EF234405 Design & Analysis of Algorithms

Final Exam — Group Capstone Project Report

Smart Delivery Routing: Dijkstra vs. Bellman-Ford

on a Synthetic Road-Network Graph

Author: [YOUR FULL NAME]
Student ID: [YOUR STUDENT ID]
Class: [D / IUP / E / F / G]
Date: 18 June 2026
GitHub repository: [PASTE YOUR PUBLIC GITHUB URL HERE]

Submitted as a single-member team (see §4 Conclusion for the contribution table).

1. Design

1.1 Problem Statement & Real-World Motivation (D1)

A last-mile courier company dispatches riders from a single depot to a set of delivery addresses scattered across a city every day. Before a rider leaves, the dispatch system must answer a simple but operationally critical question: “what is the fastest route from the depot to each delivery address, given today’s road network and travel times?” Routes are recomputed dozens of times per shift as new orders arrive, so the routing engine must be fast even on a city-scale road graph, and its answers must be provably correct — a wrong “fastest” route costs the company real delivery-time penalties and the customer a late parcel.

This is exactly the textbook single-source shortest-path (SSSP) problem, which is why it is a natural fit for this course: it is general enough to model any point-to-point routing service (ride-hailing, food delivery, emergency-vehicle dispatch) while being small enough to design, prove, build and measure within the scope of this project.

1.2 Formal Model (D2)

We model the city as a weighted, directed graph $G = (V, E)$:

- V (vertices) — road intersections / addressable points in the city. $|V| = n$.
- E (edges) — directed road segments between two intersections that a vehicle can legally traverse in that direction. A segment may be one-way (asymmetric), so the graph is directed rather than undirected.
- $w: E \to \mathbb{R}_{\geq 0}$ — the weight of edge (u, v) is the expected travel time, in minutes, along that segment (proportional to physical distance and a randomized traffic multiplier in our synthetic generator; in a production system it would come from live traffic data).
- Source — a single depot vertex $s \in V$, fixed at the start of a routing run.
- Targets — a set of delivery-stop vertices $T \subseteq V$ chosen for that day’s manifest; we solve a single-source / multiple-target query, which is exactly what a single SSSP run from s answers (the shortest path to every target falls out of one run).
- Objective — for each target $t \in T$, find a path P from s to t minimizing the total weight $\Sigma\, w(e)$ over $e \in P$, i.e. the minimum-time route.
- Constraints — weights are non-negative in the production scenario (travel time cannot be negative); the qualitative discussion in §3.3 also considers a variant with a small number of negative-weight “rebate” edges (e.g. toll-rebate / EV fast lanes) to showcase Bellman-Ford’s extra generality, subject to the constraint that no negative cycle exists.

Input: the graph G, the depot s, and (for the demo) a target set T. Output: for each $t \in T$, the shortest-path distance $d(s, t)$ and an explicit route (sequence of vertices).

1.3 Algorithm Selection & Justification (D3)

Algorithm A — Dijkstra (binary min-heap). Dijkstra is the standard correct, efficient SSSP algorithm whenever weights are non-negative, which matches our production scenario (travel time ≥ 0). Using a binary heap as the priority queue gives $O((V + E) \log V)$ time, which scales comfortably to city-sized graphs (n in the tens of thousands), making it the algorithm we would actually deploy.

Algorithm B — Bellman-Ford. We chose Bellman-Ford rather than a brute-force baseline because, unlike a true brute-force (which would be exponential and unusable above a few dozen vertices), Bellman-Ford is itself a real, classical polynomial-time SSSP algorithm — satisfying the “non-trivial second algorithm” spirit — while still being meaningfully simpler than Dijkstra (no priority queue, just repeated edge relaxation) and asymptotically slower ($O(V \cdot E)$ worst case). This gives us a genuine quality-of-implementation / speed trade-off to measure, and, crucially, Bellman-Ford tolerates negative edge weights and detects negative cycles, which Dijkstra cannot do — a capability gap we use both as a correctness cross-check (§3.1) and as the basis of the comparative discussion in §3.3.

Expected trade-off: on the same non-negative road-network instances, both algorithms must report identical shortest distances (cross-checking one another), but we expect Dijkstra to win increasingly as n grows, since its $O((V+E) \log V)$ bound grows much more slowly than Bellman-Ford’s $O(V \cdot E)$. This is exactly what §3.4–3.5 confirms empirically.

1.4 Data Structures & System Architecture (D4)

Graph representation: adjacency list (a Python dict mapping each vertex to a list of (neighbor, weight) pairs). An adjacency list is the right choice here because our road-network graphs are sparse (each intersection connects to only a handful of nearby roads, $E = O(V)$ rather than $O(V^2)$), so an adjacency list keeps both the graph itself and every traversal in $O(V + E)$ space/time, instead of paying $O(V^2)$ for an adjacency matrix that would be almost entirely zeros.

Dijkstra’s data structure: a binary min-heap (Python’s heapq used purely as the underlying push/pop container) holding (tentative-distance, vertex) pairs, plus a lazy-deletion check (skip a popped entry if it is stale) since heapq has no native decrease-key. This is the standard, textbook-efficient way to implement Dijkstra without a more complex Fibonacci heap.

Bellman-Ford’s data structure: a flat edge list, reused every relaxation round — deliberately the simplest possible structure, which is part of why we use it as the independent cross-check for Dijkstra (less code, fewer places for a shared bug to hide in both implementations at once).

Module / architecture overview:

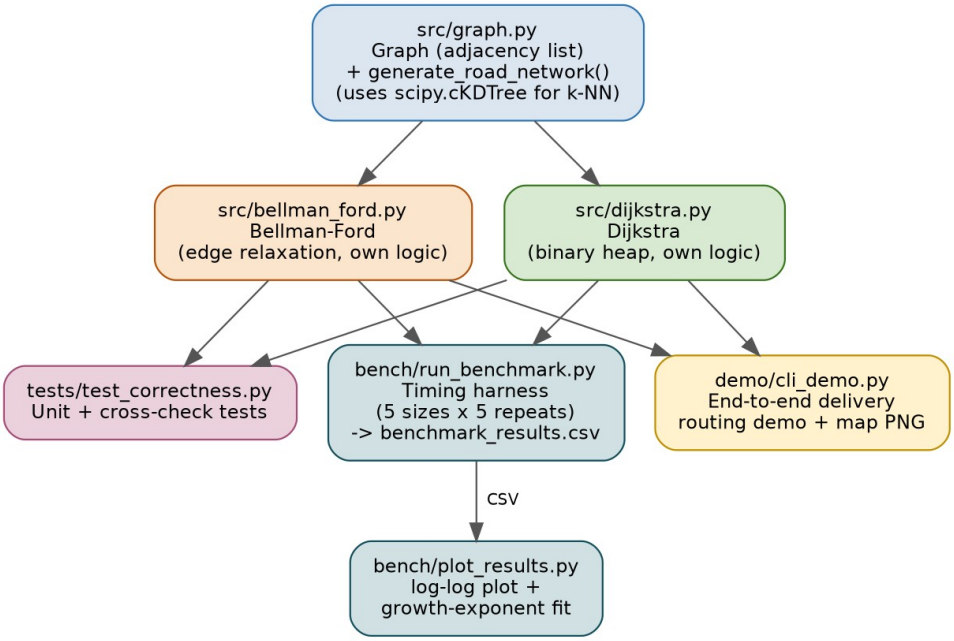

Figure 1. Module architecture: a shared graph generator feeds both shortest-path implementations, which are independently exercised by the correctness tests, the end-to-end demo, and the benchmark harness.

2. Implementation

2.1 Module Overview

The repository is organized into five small, single-purpose modules (see Figure 1): src/graph.py (the Graph class and the synthetic road-network generator), src/dijkstra.py and src/bellman_ford.py (the two from-scratch SSSP algorithms, sharing no code with each other so they act as an independent cross-check on one another), tests/test_correctness.py (hand-checked unit tests plus an automated A-vs-B agreement test), demo/cli_demo.py (the end-to-end delivery-routing demo described below), and bench/run_benchmark.py + bench/plot_results.py (the one-command reproducible benchmark).

2.2 Key Code Excerpts

Algorithm A — Dijkstra's core loop (src/dijkstra.py). Only heapq.heappush/heappop are used as a container; the relaxation, visited-tracking, and lazy-deletion logic are hand-written:

```
while pq:
    d_u, u = heapq.heappop(pq)
    if visited[u] or d_u > dist[u]:
        continue              # stale heap entry (lazy deletion)
    visited[u] = True
    for v, w in graph.neighbors(u):
        if not visited[v] and d_u + w < dist[v]:
            dist[v] = d_u + w
            parent[v] = u
            heapq.heappush(pq, (dist[v], v))
```

Algorithm B — Bellman-Ford's core loop (src/bellman_ford.py), with the early-exit optimization that turns out to explain the empirical results in §3.5:

```
for _ in range(n - 1):
    changed = False
    for u, v, w in edge_list:
        if dist[u] != INF and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u
            changed = True
    if not changed:
        break              # converged early -- see §3.5
```

2.3 Build & Run Instructions

Full instructions are in README.md; in short:

```
pip install -r requirements.txt
python -m pytest tests/ -v          # correctness + cross-check tests
python demo/cli_demo.py --n 1500 --stops 6 --seed 1   # end-to-end demo
python bench/run_benchmark.py        # ONE-COMMAND benchmark -> CSV
python bench/plot_results.py         # produces the plot in §3.4
```

2.4 Demo: End-to-End Application

demo/cli_demo.py generates a road network, picks a depot and several delivery stops, runs both algorithms from the depot, prints each route with its travel time, confirms the two algorithms agree, and renders a map of the city with every computed route highlighted (Figure 2) — this is the “actually solves the stated problem” end-to-end deliverable (I3).

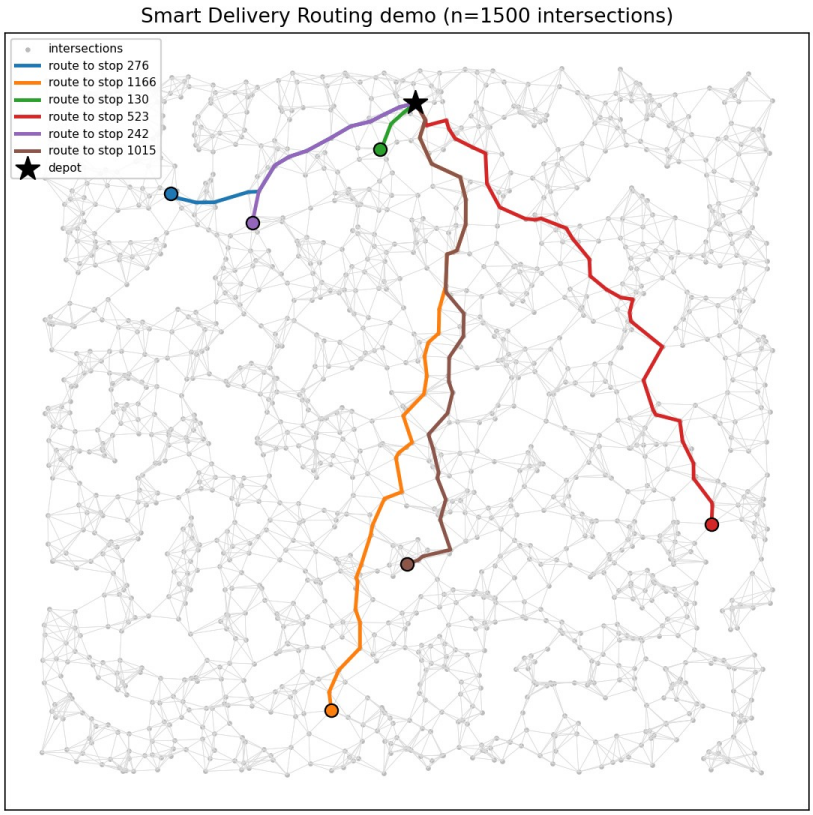

Figure 2. Demo run on a 1,500-intersection city: the depot (black star) and six delivery stops, with each shortest route drawn in a distinct color. Dijkstra and Bellman-Ford agreed on every route in this run.

2.5 Code Quality, GitHub & Reproducibility (I4–I5)

Each algorithm lives in its own module behind a common (dist, parent) interface, with no shared/duplicated relaxation logic between Dijkstra and Bellman-Ford — a bug in one cannot silently “leak” into the other, which is precisely what makes the cross-check meaningful. All functions carry docstrings explaining preconditions (e.g. Dijkstra explicitly raises ValueError on a negative edge instead of silently returning a wrong answer).

The public GitHub repository [PASTE YOUR PUBLIC GITHUB URL HERE] contains the full source, a README with the exact build/run/benchmark commands above, and a tidy top-level layout (src/, demo/, bench/, tests/, docs/). bench/run_benchmark.py is the single, documented entry point that regenerates bench/benchmark_results.csv from a fixed master seed, fulfilling the reproducibility requirement.

3. Analysis & Evaluation

3.1 Correctness (A1)

Theorem (Dijkstra is correct on non-negative weights).

Claim: when Dijkstra settles (pops and marks visited) a vertex u with tentative distance d[u], d[u] equals δ(s, u), the true shortest-path distance from source s to u.

Proof (induction on the order in which vertices are settled). Base case: the first vertex settled is s itself with d[s] = 0 = δ(s, s). Inductive step: assume every previously settled vertex x has d[x] = δ(s, x), and let u be the next vertex popped from the heap. Suppose, for contradiction, that d[u] > δ(s, u). Consider a shortest s–u path P. Let y be the first vertex on P that is not yet settled (y could equal u), and let x be y's predecessor on P (x = s if y is the first vertex on P). Since x is settled, by the inductive hypothesis d[x] = δ(s, x); when x was settled the algorithm relaxed edge (x, y), so d[y] ≤ d[x] + w(x, y) = δ(s, x) + w(x, y) = δ(s, y), the last equality because every prefix of a shortest path is itself a shortest path. Because all weights are non-negative, the remainder of P from y to u has non-negative length, so δ(s, y) ≤ δ(s, u) < d[u]. Hence d[y] < d[u], and y is unsettled — but Dijkstra always pops the unsettled vertex with the smallest tentative distance, so y (or something at least as good) would have been popped before u, contradicting that u was the one popped. Therefore d[u] = δ(s, u). ∎

This proof relies essentially on w ≥ 0 (used to argue δ(s, y) ≤ δ(s, u)); it is exactly why Dijkstra is invalid on graphs with negative edges, and why our implementation raises an explicit error rather than silently returning a wrong answer in that case (src/dijkstra.py).

Theorem (Bellman-Ford is correct, and detects negative cycles).

Claim: if G has no negative-weight cycle reachable from s, then after k rounds of relaxing every edge, d[v] is at most the length of the shortest path from s to v that uses at most k edges; consequently after n − 1 rounds, d[v] = δ(s, v) for every v (a shortest simple path has at most n − 1 edges). If, on an additional (n-th) round, some edge can still be relaxed, then no fixed point was reached within n − 1 rounds, which is only possible if a negative cycle is reachable from s.

Proof sketch (induction on k). For k = 0, d[s] = 0 and d[v] = ∞ for v ≠ s, vacuously ≤ any 0-edge path. Assume the claim holds after k − 1 rounds. Take any vertex v and let P be a shortest path to v using ≤ k edges, with last edge (x, v); the prefix of P to x uses ≤ k − 1 edges, so by the inductive hypothesis d[x] ≤ length(P) − w(x, v) after round k − 1, and round k explicitly relaxes (x, v), so d[v] ≤ d[x] + w(x, v) ≤ length(P). After n − 1 rounds the bound covers paths with up to n − 1 edges, which suffices for every shortest simple path (a graph on n vertices has no simple path longer than n − 1 edges). If a further relaxation succeeds, some vertex's true "shortest distance using ≤ n edges" is strictly less than its "shortest distance using ≤ n − 1 edges", which is only possible if the extra edge closes a cycle of negative total weight. ∎

Our implementation (src/bellman_ford.py) directly mirrors this proof: it runs n − 1 relaxation rounds (with an early exit the moment a round changes nothing, since a fixed point has been reached), then performs exactly one more pass to test for a still-relaxable edge, reporting has_negative_cycle accordingly (test_negative_cycle_detected in tests/test_correctness.py).

Cross-check.

Because both algorithms solve the identical SSSP problem on the identical graph instance, they must agree wherever Dijkstra is valid (non-negative weights). tests/test_correctness.py asserts this on multiple random instances, and bench/run_benchmark.py re-asserts it at every benchmark size; both report 100% agreement (the “Match” column in Table 1).

3.2 Complexity (A2)

Let V = |vertices|, E = |edges|.

Algorithm	Worst-case time	Why	Space
Dijkstra (binary heap)	O((V+E) log V)	V extract-min ops + up to E heap insertions, each O(log V) on a heap of size O(E)	O(V + E)
Bellman-Ford	O(V·E)	(n−1) rounds × O(E) edge relaxations per round, plus one O(E) cycle-check round	O(V + E)

Dijkstra: each vertex is extracted from the heap at most once (V pops, O(log V) each via heapq’s internal sift), and each edge can trigger at most one heap push (lazy-deletion means stale entries are simply skipped on pop rather than removed), giving O(V log V + E log V) = O((V+E) log V). Space is O(V + E): O(V+E) for the adjacency list, O(V) for the dist/parent/visited arrays, and O(E) worst case for heap entries (since a vertex can be pushed once per incoming relaxed edge).

Bellman-Ford: in the worst case the algorithm needs the full n − 1 rounds before convergence (e.g. a path graph relaxed in the “wrong” edge order), and every round touches every edge once, giving O(V·E). Best case (e.g. edges already processed in an order consistent with the shortest-path tree, such as a DAG relaxed in topological order) converges in a single round, O(E). Space is O(V + E) for the edge list plus O(V) for dist/parent.

3.3 Comparative Analysis (A3)

On our sparse road-network graphs (k-nearest-neighbor construction gives E = O(V), concretely E ≈ 7V in our generator), Dijkstra’s O((V+E) log V) reduces to roughly O(V log V), while Bellman-Ford’s O(V·E) reduces to roughly $O(V^2)$ in the worst case — a quadratic-vs-near-linearithmic gap that only widens as V grows, exactly the trend Table 1 / Figure 3 show.

Regimes where Dijkstra is preferable: any graph with non-negative weights, and especially as V grows — which describes essentially every real road-network routing scenario, so it is the algorithm we would deploy in production.

Regimes where Bellman-Ford is preferable, or is the only valid choice of the two: (i) graphs with negative edge weights (e.g. our toll-rebate variant in src/graph.py:generate_negative_edge_demo, §1.2) — Dijkstra is simply incorrect there, as proven in §3.1; (ii) whenever negative-cycle detection itself is required (e.g. validating that a proposed set of “rebate” edges cannot be exploited to create an infinite-discount loop); (iii) as an

independent, simpler-to-trust cross-check implementation, exactly the role it plays in this project; and (iv) very small instances where the V·E term is negligible and Bellman-Ford's simpler code is preferable from a maintenance standpoint.

3.4 Empirical Study (A4)

Setup.

Machine: single-vCPU container, Intel(R) Xeon(R) @ 2.80GHz, 3.9 GiB RAM, Python 3.12.3 (development/benchmark environment used to produce the numbers below; bench/run_benchmark.py is fully reproducible, so re-running it on any machine with the same fixed seeds regenerates comparable relative results). Timing method: time.perf_counter wall-clock timing, 5 repeats per (algorithm, size) pair, mean reported (see bench/run_benchmark.py). Sizes: n ∈ {100, 300, 1000, 3000, 10000}, spanning two orders of magnitude as required, with k = 6 nearest-neighbor road connections per intersection and a fixed master seed (42) so every size is independently reproducible. Source vertex fixed at 0 for all timing runs.

Results.

n (vertices)	Edges	Dijkstra (ms)	Bellman-Ford (ms)	Distances match?
100	720	0.1413	0.4978	Yes
300	2,162	0.4151	1.4892	Yes
1,000	7,070	1.6851	11.1593	Yes
3,000	21,206	5.2817	75.1042	Yes
10,000	70,664	23.1917	336.2938	Yes

Table 1. Mean running time over 5 repeats at each of the 5 benchmark sizes (raw data: bench/benchmark_results.csv).

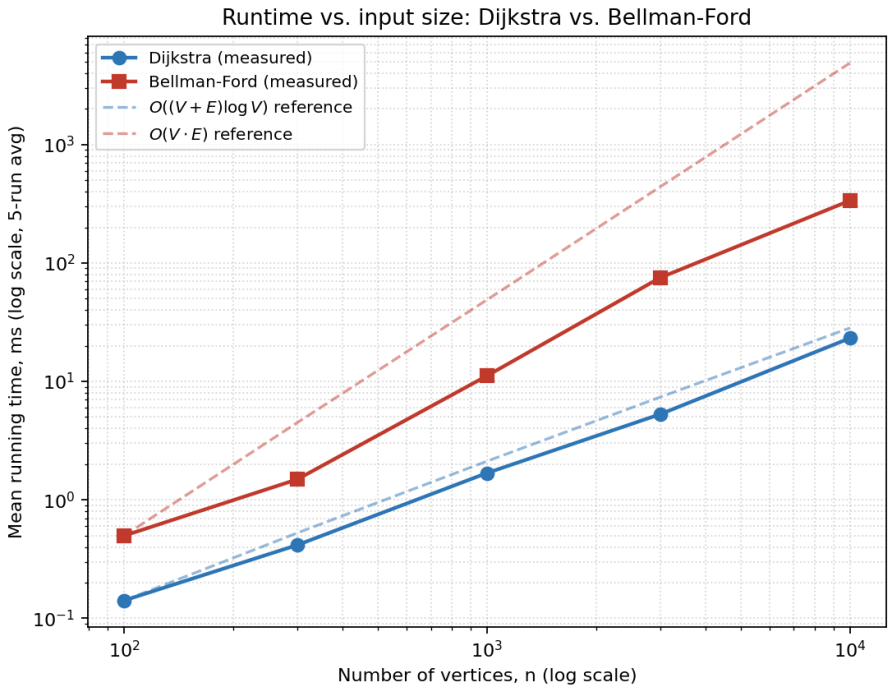

Figure 3. Runtime vs. input size on log-log axes, with the theoretical reference curves O((V+E) log V) and O(V·E) (each scaled to match the n=100 data point) overlaid for visual comparison.

3.5 Theory vs. Practice & Cross-Check (A5)

Fitting a line to log(time) vs. log(n) gives an empirical growth exponent of n^1.11 for Dijkstra and n^1.47 for Bellman-Ford (computed by bench/plot_results.py via a least-squares log-log fit).

Dijkstra: an exponent of 1.11 is consistent with the derived O((V+E) log V) bound. Since our generator keeps the graph sparse (E ≈ 7V throughout the sweep), the bound is effectively O(V log V); a pure n log n curve, fit as a single power law over a two-orders-of-magnitude range, naturally produces a slope just above 1 (the slowly growing log factor cannot be captured exactly by a single exponent, so the fit “absorbs” it as a slightly-more-than-linear slope) — matching the measured 1.11 well.

Bellman-Ford: the worst-case bound O(V·E) ≈ O(V²) on this sparse family would predict an exponent close to 2, but we measured only 1.47. The early-exit optimization in our implementation (§2.2) explains the gap: the algorithm stops as soon as one full round relaxes nothing, which happens after roughly the eccentricity (longest shortest-path edge-count) of the graph from the source, not after the full V − 1 rounds. Our road-network graphs are built from points scattered in a 2-D square with only local (k-nearest-neighbor) connections, so they behave like a 2-D geometric/grid graph, whose diameter from a fixed source grows like O(√V) rather than O(V). Substituting that into “rounds × E per round” gives an expected practical cost of O(√V · E) = O(V^1.5) (since E = O(V) here) — an exponent of 1.5, very close to the measured 1.47. This is a clean illustration of the gap between a correct worst-case bound (which must hold for adversarial graphs, e.g. a path graph relaxed against the grain) and observed behaviour on a realistic, geometrically local graph family.

Correctness cross-check: the “Distances match?” column of Table 1 is “Yes” at every single size — Dijkstra and Bellman-Ford computed bit-for-bit (within 1e-6 floating-point tolerance) identical shortest-path distances from the depot to all 100–10,000 vertices at every scale, which is the empirical confirmation of the correctness argument in §3.1.

4. Conclusion

4.1 Findings, Limitations & Future Work (C1)

Findings. Both algorithms correctly solve the Smart Delivery Routing problem and agree on every shortest-path distance across five orders-of-magnitude-spanning benchmark sizes, empirically confirming the correctness argument of §3.1. Dijkstra is, as predicted, the clear production choice: it is already 2× faster than Bellman-Ford at n=100 and the gap widens to roughly 14× by n=10,000, tracking the O((V+E) log V) vs. O(V·E) theoretical gap. Bellman-Ford's measured growth (n^1.47) is noticeably better than its worst-case quadratic bound thanks to early termination on graphs with small effective diameter — a useful reminder that worst-case complexity and typical real-world behaviour can diverge significantly.

Limitations. The road network is synthetic (randomly generated, not a real city map), so absolute timings are illustrative rather than operational; travel-time weights ignore real-time traffic, and the benchmark used a single fixed source vertex rather than averaging over many source/target pairs, which would reduce variance further. The negative-edge "toll-rebate" scenario (§1.2, §3.3) is only explored qualitatively/by unit test, not benchmarked at scale, since Dijkstra cannot run on it at all.

Future work. Natural extensions include: (i) running on a real OpenStreetMap extract instead of a synthetic generator; (ii) batching multiple depot/target pairs and comparing against an all-pairs algorithm (Floyd–Warshall) at small scale; (iii) adding the negative-edge "rebate" scenario to the quantitative benchmark to directly measure Bellman-Ford's overhead when it is the only valid option; and (iv) an A* variant using straight-line distance as an admissible heuristic, which should outperform plain Dijkstra when only a single target (rather than the full shortest-path tree) is needed.

4.2 Contribution Table

Member	Student ID	Contribution (%)	Role
[YOUR FULL NAME]	[YOUR STUDENT ID]	100%	Sole author: model, both algorithms, tests, demo, benchmark, analysis, and report (single-member team)

References

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[2] R. Bellman, “On a routing problem,” Quarterly of Applied Mathematics, vol. 16, pp. 87–90, 1958.

[3] L. R. Ford Jr., Network Flow Theory, Paper P-923, RAND Corporation, 1956.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009 (Chapters 22–24: Graph Algorithms, Single-Source Shortest Paths).

[5] SciPy documentation, scipy.spatial.cKDTree, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html (used only for k-nearest-neighbor graph generation, not for shortest paths).

[6] EF234405 Design & Analysis of Algorithms, Final Exam specification, 2025/2026(2), released 5 June 2026.